

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Technical Presentation

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 2 – Technical Presentation
Weightage:	30% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	k.kalyan	Reg. No.:	192425301
Section / Batch:	Ai&ml	Date:	

Code of Conduct

I K.Kalyan(192425301) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Suggested Team Member Roles (3 Members)
- Member 1: Theory, Concepts, and Literature Review
- Member 2: Algorithm Design, Models, and Technical Analysis
- Member 3: Demonstration, Case Study, Applications, and Conclusion
- GPS photo must submit.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Evolution of Natural Language Processing: From Rule-Based Systems to Modern NLP	NLP Models, Language Processing Evolution	TP-01
Comparative Study of NLP Models and Algorithms	Model Analysis, Algorithm Comparison	TP-02
Regular Expressions for Real-World Text Processing	Pattern Matching, Information Extraction	TP-03
Designing Efficient Regular Expression Patterns for Information Extraction	Text Mining, Data Extraction Techniques	TP-04
Finite State Automata in Natural Language Processing	State Transitions, Language Recognition	TP-05
Building a Finite State Automaton for Token Recognition	Lexical Analysis, Token Validation	TP-06
Morphology and Word Formation in Natural Languages	Morpheme Analysis, Word Structure Processing	TP-07
Inflectional Morphology: Understanding Grammatical Variations	Grammatical Analysis, Word Normalization	TP-08
Derivational Morphology and Vocabulary Expansion	Word Generation, Semantic Transformation	TP-09
Comparative Analysis of Inflectional and Derivational Morphology	Morphological Classification, Language Analysis	TP-10

Finite-State Morphological Parsing Techniques	Morphological Parsing, Finite-State Processing	TP-11
Morphological Analysis for Indian Languages	Language-Specific Morphology, NLP Challenges	TP-12
Porter Stemmer: Design, Working Mechanism, and Applications	Stemming Algorithms, Text Preprocessing	TP-13
Stemming vs Morphological Parsing: A Comparative Study	Word Reduction, Morphological Processing	TP-14
Integrated NLP Pipeline: From Regular Expressions to Morphological Processing	End-to-End Language Processing Pipeline	TP-15

Annexure A – Technical Presentation

Team No.	Technical Presentation Title	Description
1	Evolution of Natural Language Processing: From Rule-Based Systems to Modern NLP	Analyze the historical development of NLP models, key milestones, and how language processing has evolved from handcrafted rules to intelligent systems.
2	Comparative Study of NLP Models and Algorithms	Explore different NLP models and algorithms used for language processing, comparing their working principles, strengths, and limitations.
3	Regular Expressions for Real-World Text Processing	Demonstrate how regular expressions are used for data validation, text extraction, log analysis, and information retrieval in practical applications.
4	Designing Efficient Regular Expression Patterns for Information Extraction	Analyze complex regular expression patterns used to extract emails, phone numbers, URLs, dates, and structured information from unstructured text.
5	Finite State Automata in Natural Language Processing	Explain the concepts of deterministic and non-deterministic finite automata and their role in lexical analysis and language recognition.
6	Building a Finite State Automaton for Token Recognition	Design and demonstrate a finite state automaton that recognizes tokens such as identifiers, keywords, and numbers in textual data.
7	Morphology and Word Formation in Natural Languages	Investigate how words are structured and formed using morphemes, and analyze the importance of morphology in NLP systems.
8	Inflectional Morphology: Understanding Grammatical Variations	Examine how inflectional changes affect word forms based on tense, number, gender, and case across different languages.
9	Derivational Morphology and Vocabulary Expansion	Analyze derivational processes that create new words and study their impact on semantic interpretation and language understanding.
10	Comparative Analysis of Inflectional and Derivational Morphology	Compare the characteristics, applications, and challenges of inflectional and derivational morphology in NLP tasks.
11	Finite-State Morphological Parsing Techniques	Explore finite-state transducers and parsing techniques used to analyze and generate morphological structures in natural languages.
12	Morphological Analysis for Indian Languages	Study the challenges and solutions involved in performing morphological processing for morphologically rich Indian languages such as Tamil, Hindi, and Telugu.
13	Porter Stemmer: Design, Working Mechanism, and Applications	Analyze the Porter Stemming algorithm, its rule-based approach, implementation steps, and role in information retrieval systems.
14	Stemming vs Morphological Parsing: A Comparative Study	Compare stemming and morphological parsing techniques based on accuracy, computational complexity, and application scenarios.
15	Integrated NLP Pipeline: From Regular Expressions to Morphological Processing	Demonstrate a complete NLP preprocessing pipeline incorporating regular expressions, finite state automata, stemming, and morphological analysis for text processing.

Rubrics for Calculation

Technical Presentation					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Team No. / Criteria Level	Technical Content Accuracy	Problem Identification	Use of Tools	Communication	Professionalism	Total %

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

</>



PATTERN-BASED TEXT PROCESSING USING REGEX

Finding, matching, and transforming text through the language of regular-expression patterns

```
re.findall(r"[A-Z][a-z]+", text)
```

WHAT IS REGEX?

A compact language for describing patterns in text

regex101

A **regular expression** (regex) is a sequence of characters that defines a search pattern.

- Instead of matching exact text, it matches a shape or structure text can take
- Interpreted by a regex engine built into nearly every programming language
- Same core syntax works across Python, JavaScript, Java, grep, and more

PATTERN → MATCH

\d{3}-\d{4}

matches a phone-style number

Call me at **555-0148** today

The engine scans left to right and reports every substring that fits the pattern's shape.

Underlying engine: NFA / DFA pattern matcher

CORE BUILDING BLOCKS

The small set of symbols that combine into any pattern

`\d` `\w` `\s`

Character classes

digit, word character, whitespace

`.` `[ ]`

Wildcards & sets

any character, or one from a set

`*` `+` `?`

Quantifiers

zero+, one+, zero-or-one repetitions

`^` `$`

Anchors

start of line, end of line

`( )`

Groups

capture or apply logic to a sub-pattern

`|`

Alternation

match one option out of several

WHERE REGEX GETS USED

Four everyday jobs for pattern-based text processing

01

Validation

Confirm input matches an expected shape — emails, phone numbers, postal codes, passwords

```
^[\\w.]+@[\\w.]+\\.\\w+$
```

02

Extraction

Pull structured values out of logs, HTML, or free text — dates, IDs, error codes

```
ERROR\\s+(\\d{3,4})
```

03

Find & Replace

Reformat or clean data at scale — renaming variables, normalizing whitespace

```
s/\\s+/ /g
```

04

Search & Tokenize

Split text into meaningful units for search engines and language processing

```
\\b\\w+\\b
```


A SIMPLE REGEX WORKFLOW

From raw text to reliable pattern matching

1

Define the shape

Describe the structure you're looking for, not the literal text



2

Build incrementally

Start simple, then add classes, quantifiers, and anchors



3

Test against real data

Check edge cases: empty strings, extra spaces, unicode



4

Apply & handle misses

Plan for text that legitimately won't match the pattern

QUICK TIPS

- Use raw strings (r"(...)") in Python so backslashes aren't double-escaped
- Watch for catastrophic backtracking with nested repeating groups
- Prefer non-greedy quantifiers (*?, +?) when matching inside delimiters
- Name capture groups (?P<name>...) to keep extraction code readable

Presentation Photo

